

HisabDo AI — Day 16

Taha — AI Recommendation Engine

Complete requirements, methodology, rules, thresholds, testing, team coordination and .NET/C# reference implementation.

1. Objective & Scope

Day 16 converts verified intelligence from Day 10–14 into structured, actionable recommendations. The MVP is deterministic and explainable. It consumes only verified Financial Health, Spending Pattern, Anomaly, Cash-Flow Forecast, Budget Status and Budget Recommendation results. The LLM is an explanation layer only.

2. Exact 17 Tasks — Coverage

1. Define and finalize Day 16 AI Recommendation Engine requirements.
2. Define the purpose and scope of the recommendation engine.
3. Define what types of financial recommendations the system should generate.
4. Define recommendation priority and severity logic.
5. Define which verified financial results will be used for recommendations.
6. Define recommendation rules using Financial Health, Spending Patterns, Anomalies, Cash-Flow Forecast, Budget Status and Budget Recommendations.
7. Define recommendation output structure.
8. Define recommendation conditions and thresholds.
9. Define how multiple recommendations will be prioritized.
10. Define insufficient-data and missing-data handling.
11. Ensure recommendations are based only on verified backend results.
12. Ensure AI does not invent financial facts or recommendations.
13. Coordinate Jaffer, Omesha, Aarti and Laiba.
14. Review backend, AI/ML and UI implementation.
15. Validate recommendations using sample financial datasets.
16. Define testing scenarios and edge cases.
17. Prepare Day 16 documentation and acceptance checklist.

3. Recommendation Types

- Spending Reduction — address verified overall/category expense growth.
- Budget Adjustment — address verified over-budget status or verified budget recommendation.
- Saving Improvement — triggered by verified low saving rate.
- Cash-Flow Risk — triggered by verified forecasted saving at or below zero.
- Unusual Spending Review — triggered only by verified anomaly/unusual pattern.
- Recurring Expense Review — triggered by verified recurring candidate.
- Financial Health Improvement — triggered by verified low health score.
- Category Spending Control — triggered by significant verified category increase.

4. Verified Inputs

Day 10: Health Score, Saving Rate, Expense Growth. Day 11: category trends, unusual/recurring flags. Day 12: anomaly type/severity/category/amount. Day 13: expected income, expense, saving, closing balance, confidence and limitations. Day 14: budget utilization/status and verified budget recommendations.

5. MVP Conditions & Thresholds

Condition	Rule	Default Priority
Low saving rate	< 10%	High
High expense growth	>= 20%	High
Category spending increase	>= 20%	Medium
High budget utilization	> 120%	Medium
Critical budget utilization	>= 150%	High
Forecast cash-flow risk	Expected saving <= 0	High
Financial health improvement	Score <= 59	High
Unusual spending	Only if verified by Day 11/12	Based on verified severity
Recurring expense	Only if verified by Day 11	Medium

Note: these are MVP engineering thresholds, not universal financial advice. Keep them configurable and validate against real HisabDo data.

6. Priority & Severity

Priority determines what is surfaced first: High, Medium, Low. Severity describes the seriousness of the verified condition: Critical, High, Medium, Low. The engine must never assign severity to an unverified event. Multiple recommendations are sorted High → Medium → Low priority, then Critical → High → Medium → Low severity, then category for deterministic output.

7. Output Structure

```
{
  "type": "BudgetAdjustment",
  "priority": "High",
  "severity": "High",
  "category": "Food",
  "reason": "Verified budget utilization is 138%",
  "action": "Review Food spending and compare it with the existing budget.",
  "verifiedAmount": 17940,
  "verifiedPercentage": 138
}
```

8. Missing / Insufficient Data

No health result → no health recommendation. No spending history → no category recommendation. No anomaly result → no anomaly recommendation. Insufficient forecast history → no forecast-based recommendation unless the forecast engine supplies a verified result. No budget → never claim over-budget or invent a budget. No opening balance → do not invent closing balance. Conflicting/empty data → return safe limitation and do not guess.

9. AI No-Invention Boundary

Backend calculations → verified Day 10–14 results → Recommendation Engine → Verified Recommendation DTO → Aarti AI Explanation → grounding validation → user. AI may explain the verified recommendation but must not create/change amounts, percentages, scores, dates, categories, forecasts, anomalies or recommendations.

10. Recommended AI Prompt

You are the explanation layer for HisabDo recommendations.
 Use ONLY verified recommendation data supplied by the backend.
 Explain why each recommendation exists and what action the user can review.
 Never invent or change amounts, percentages, scores, dates, categories, thresholds, forecasts, anomalies or recommendations.
 If a value is missing, do not guess. Do not use external financial data.
 Communicate limitations and uncertainty supplied by the backend.

11. Complete .NET/C# Reference Implementation

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace HisabDo.AI.Day16;
public enum RecommendationType
{
    SpendingReduction, BudgetAdjustment, SavingImprovement, CashFlowRisk,
```

```

        UnusualSpendingReview, RecurringExpenseReview,
        FinancialHealthImprovement, CategorySpendingControl
    }
    public enum RecommendationPriority { High, Medium, Low }
    public enum RecommendationSeverity { Critical, High, Medium, Low }
    public sealed record VerifiedRecommendationInput
    {
        public required string UserId { get; init; }
        public double? FinancialHealthScore { get; init; }
        public double? SavingRatePercent { get; init; }
        public double? ExpenseGrowthPercent { get; init; }
        public IReadOnlyList<VerifiedSpendingPattern> SpendingPatterns { get; init; } = Array.Empty<VerifiedSpendingPattern>();
        public IReadOnlyList<VerifiedAnomaly> Anomalies { get; init; } = Array.Empty<VerifiedAnomaly>();
        public double? ExpectedIncome { get; init; }
        public double? ExpectedExpense { get; init; }
        public double? ExpectedSaving { get; init; }
        public double? ExpectedClosingBalance { get; init; }
        public string? ForecastConfidence { get; init; }
        public IReadOnlyList<VerifiedBudgetStatus> BudgetStatuses { get; init; } = Array.Empty<VerifiedBudgetStatus>();
        public IReadOnlyList<VerifiedBudgetRecommendation> BudgetRecommendations { get; init; } = Array.Empty<VerifiedBudgetRecommendation>();
        public IReadOnlyList<string> Limitations { get; init; } = Array.Empty<string>();
    }
    public sealed record VerifiedSpendingPattern
    {
        public required string Category { get; init; }
        public double CurrentAmount { get; init; }
        public double? PercentageChange { get; init; }
        public bool IsUnusual { get; init; }
        public bool IsRecurringCandidate { get; init; }
    }
    public sealed record VerifiedAnomaly
    {
        public required string Type { get; init; }
        public required string Severity { get; init; }
        public required string Category { get; init; }
        public double Amount { get; init; }
    }
    public sealed record VerifiedBudgetStatus
    {
        public required string Category { get; init; }
        public double ActualExpense { get; init; }
        public double? UtilizationPercent { get; init; }
        public bool IsOverBudget { get; init; }
    }
    public sealed record VerifiedBudgetRecommendation
    {
        public required string Category { get; init; }
        public double RecommendedBudget { get; init; }
        public string Reason { get; init; } = "";
    }
    public sealed record Recommendation
    {
        public required RecommendationType Type { get; init; }
        public required RecommendationPriority Priority { get; init; }
        public required RecommendationSeverity Severity { get; init; }
        public required string Category { get; init; }
        public required string Reason { get; init; }
        public required string Action { get; init; }
        public double? VerifiedAmount { get; init; }
        public double? VerifiedPercentage { get; init; }
    }
    public sealed record RecommendationResult
    {
        public required string UserId { get; init; }
        public IReadOnlyList<Recommendation> Recommendations { get; init; } = Array.Empty<Recommendation>();
        public IReadOnlyList<string> Limitations { get; init; } = Array.Empty<string>();
    }
    /// <summary>
    /// Deterministic MVP engine. It consumes only verified Day 10-14 results.
    /// Keep thresholds configurable in production.
    /// </summary>
    public sealed class TahaRecommendationEngine
    {
        private const double LowSavingRate = 10.0;
        private const double HighExpenseGrowth = 20.0;
        private const double HighBudgetUtilization = 120.0;
        private const double CriticalBudgetUtilization = 150.0;
        public RecommendationResult Generate(VerifiedRecommendationInput input)
        {
            if (input is null) throw new ArgumentNullException(nameof(input));
            if (string.IsNullOrEmpty(input.UserId))
                throw new InvalidOperationException("Authenticated UserId is required.");
            var r = new List<Recommendation>();
            if (input.FinancialHealthScore is <= 59)
            {
                r.Add(new Recommendation {
                    Type=RecommendationType.FinancialHealthImprovement, Priority=RecommendationPriority.High,
                    Severity=RecommendationSeverity.High, Category="Overall",
                    Reason="$Verified Financial Health Score is {input.FinancialHealthScore}/100.",
                    Action="Review the verified weak financial factors and address the highest-impact area.",
                    VerifiedPercentage=input.FinancialHealthScore
                });
            }
            if (input.SavingRatePercent is < LowSavingRate)
            {
                r.Add(new Recommendation {
                    Type=RecommendationType.SavingImprovement, Priority=RecommendationPriority.High,

```

```

        Severity=RecommendationSeverity.High, Category="Savings",
        Reason=$"Verified saving rate is {input.SavingRatePercent}%, below the MVP threshold of {LowSavingRate}%.",
        Action="Review high-spending categories and identify a practical saving opportunity.",
        VerifiedPercentage=input.SavingRatePercent
    });
    if (input.ExpenseGrowthPercent is >= HighExpenseGrowth)
    {
        r.Add(new Recommendation {
            Type=RecommendationType.SpendingReduction, Priority=RecommendationPriority.High,
            Severity=RecommendationSeverity.High, Category="Overall Expenses",
            Reason=$"Verified expense growth is {input.ExpenseGrowthPercent}%, meeting the {HighExpenseGrowth}% threshold.",
            Action="Review categories with the largest verified increases.",
            VerifiedPercentage=input.ExpenseGrowthPercent
        });
    }
    foreach (var p in input.SpendingPatterns)
    {
        if (p.IsUnusual)
        {
            r.Add(new Recommendation {
                Type=RecommendationType.UnusualSpendingReview, Priority=RecommendationPriority.High,
                Severity=RecommendationSeverity.High, Category=p.Category,
                Reason=$"Verified spending engine marked {p.Category} as unusual.",
                Action="Review the recorded transactions in this category.",
                VerifiedAmount=p.CurrentAmount, VerifiedPercentage=p.PercentageChange
            });
        }
        if (p.IsRecurringCandidate)
        {
            r.Add(new Recommendation {
                Type=RecommendationType.RecurringExpenseReview, Priority=RecommendationPriority.Medium,
                Severity=RecommendationSeverity.Medium, Category=p.Category,
                Reason=$"Verified pattern engine identified a recurring-expense candidate in {p.Category}.",
                Action="Review whether this recurring expense is still necessary and within budget.",
                VerifiedAmount=p.CurrentAmount
            });
        }
        if (p.PercentageChange is >= 20)
        {
            r.Add(new Recommendation {
                Type=RecommendationType.CategorySpendingControl, Priority=RecommendationPriority.Medium,
                Severity=RecommendationSeverity.Medium, Category=p.Category,
                Reason=$"Verified spending in {p.Category} increased by {p.PercentageChange}%.",
                Action="Review the transactions driving this increase and consider spending controls.",
                VerifiedAmount=p.CurrentAmount, VerifiedPercentage=p.PercentageChange
            });
        }
    }
    foreach (var a in input.Anomalies)
    {
        bool high = a.Severity.Equals("High", StringComparison.OrdinalIgnoreCase) ||
            a.Severity.Equals("Critical", StringComparison.OrdinalIgnoreCase);
        r.Add(new Recommendation {
            Type=RecommendationType.UnusualSpendingReview,
            Priority=high ? RecommendationPriority.High : RecommendationPriority.Medium,
            Severity=high ? RecommendationSeverity.High : RecommendationSeverity.Medium,
            Category=a.Category,
            Reason=$"Verified anomaly: {a.Type} ({a.Severity}).",
            Action="Review the transaction details and confirm whether the spending is expected.",
            VerifiedAmount=a.Amount
        });
    }
}
if (input.ExpectedSaving is <= 0)
{
    r.Add(new Recommendation {
        Type=RecommendationType.CashFlowRisk, Priority=RecommendationPriority.High,
        Severity=RecommendationSeverity.High, Category="Cash Flow",
        Reason=$"Verified forecasted saving is {input.ExpectedSaving}.",
        Action="Review expected expenses and upcoming commitments before the forecast period.",
        VerifiedAmount=input.ExpectedSaving
    });
}
foreach (var b in input.BudgetStatuses.Where(x => x.IsOverBudget))
{
    double u = b.UtilizationPercent ?? 0;
    bool critical = u >= CriticalBudgetUtilization;
    r.Add(new Recommendation {
        Type=RecommendationType.BudgetAdjustment,
        Priority=critical ? RecommendationPriority.High : RecommendationPriority.Medium,
        Severity=critical ? RecommendationSeverity.High : RecommendationSeverity.Medium,
        Category=b.Category,
        Reason=$"Verified budget utilization for {b.Category} is {u}%",
        Action="Review spending in this category and compare it with the existing budget.",
        VerifiedAmount=b.ActualExpense, VerifiedPercentage=u
    });
}
foreach (var b in input.BudgetRecommendations)
{
    r.Add(new Recommendation {
        Type=RecommendationType.BudgetAdjustment, Priority=RecommendationPriority.Low,
        Severity=RecommendationSeverity.Low, Category=b.Category,
        Reason=$"Verified budget engine recommends {b.RecommendedBudget} for {b.Category}. {b.Reason}",
        Action="Review the recommended budget against your current financial plan.",
        VerifiedAmount=b.RecommendedBudget
    });
}
var ordered = r.OrderBy(x => PriorityRank(x.Priority))
    .ThenBy(x => SeverityRank(x.Severity))
    .ThenBy(x => x.Category, StringComparer.OrdinalIgnoreCase)
    .ToList();
return new RecommendationResult {
    UserId=input.UserId, Recommendations=ordered, Limitations=input.Limitations
};
};

private static int PriorityRank(RecommendationPriority p) => p switch {
    RecommendationPriority.High => 1, RecommendationPriority.Medium => 2, _ => 3
};

```

```

};
private static int SeverityRank(RecommendationSeverity s) => s switch {
    RecommendationSeverity.Critical => 1, RecommendationSeverity.High => 2,
    RecommendationSeverity.Medium => 3, _ => 4
};
}

```

12. Team Coordination

Jaffer: connect verified Day 10–14 results, UserId isolation, recommendation API, missing-data handling.

Omesha: validate methodology, thresholds, rule consistency, sample datasets and future ML direction.

Aarti: explain verified recommendations, enforce AI grounding/no-invention rules and prepare samples.

Laiba: integrate API into mobile/web UI and show priority, severity, reason, action and limitations.

Taha: finalize requirements, review all layers, validate results, coordinate blockers and approve acceptance.

13. Sample Validation Dataset

Verified Input	Sample	Expected
Financial Health	55/100	Health improvement
Saving Rate	6%	Saving improvement
Expense Growth	25%	Spending reduction
Food category change	28%	Category control
Food anomaly	High	Unusual spending review
Expected saving	-5000	Cash-flow risk
Food budget utilization	138%	Budget adjustment

14. Testing & Edge Cases

1. Test values exactly at and around 10%, 20%, 120% and 150% thresholds.
2. Test zero/negative expected saving.
3. Test missing health, spending, anomaly, budget and forecast results.
4. Test insufficient historical data.
5. Test multiple recommendations and deterministic ordering.
6. Test duplicate recommendations.
7. Test User A cannot receive User B's verified results.
8. Test extreme values and null fields.
9. Test empty input.
10. Test AI attempt to invent/change a financial number.
11. Test AI attempt to create an unsupported recommendation.
12. Validate recommendations across multiple datasets and review false positives/negatives.

15. Assumptions & Limitations

Day 10–14 outputs are assumed to be verified and user-scoped. MVP thresholds are configurable engineering defaults and must be validated on representative HisabDo data. Recommendations are informational/action-oriented and do not guarantee outcomes. Advanced ML ranking/personalization should be considered only after sufficient real-data evaluation.

16. Acceptance Checklist

- All 17 Taha tasks are covered.
- Recommendation types, priority and severity are finalized.
- Day 10–14 verified inputs are mapped.

- Conditions and thresholds are documented.
- Multiple recommendations are prioritized deterministically.
- Missing/insufficient data is handled safely.
- No-invention boundary is defined.
- Team responsibilities are clear.
- Sample datasets and edge cases are defined.
- C# reference implementation is ready for integration.

17. Final Definition

Day 16 takes verified financial intelligence from Day 10–14 and converts it into structured, prioritized and explainable actions. Backend remains the source of truth; the recommendation engine applies transparent rules; AI only explains the verified recommendation. Actual HisabDo entities, repositories, authentication and API contracts must be mapped before production.